

m labs

SAYMTHEON MERCURY

HALF-DEVELOP REPORT

(Early report)

SAYMTHEON

MERCURY

HALF-DEVELOPING REPORT

Early Development Report | September 2026

Internal development document

Developed by SAYMTHEON.

1. EXECUTIVE SUMMARY

SAYMTHEON MERCURY is a 7B-class development model representing the transition toward a more general-purpose and production-oriented SAYMTHEON model line. This document is not a final product report; it is a half-developing report documenting the current checkpoint, benchmark results, and known behavioral gaps.

Current results show meaningful progress in core capability and system behavior. However, exact instruction following, mathematics/reasoning response mode, false-premise handling, agent boundaries, and response-length control still require calibration.

Current status: MERCURY is runnable, testable, and suitable for continued development, but it should not yet be treated as a final production release. The next phase should focus on behavioral calibration while preserving the capabilities already gained.

Area	Value
Model class	7B class / 7.248B parameters
Base model	Mistral-7B-v0.3
Training approach	Full-parameter BF16 SFT
Training hardware	NVIDIA H200
Valid training examples	55,278
Selected checkpoint	checkpoint-3250
Best evaluation loss	0.5813
Main benchmark	1,500 records / 1,286 unique prompts
Status	Half-developing / calibration pending

2. REPORT SCOPE

This report follows the same engineering principle used in the previous SAYMTHEON LITE 4B development work: a model is not evaluated by loss alone. Behavioral benchmarks, capability, boundary behavior, and response quality are considered together.

The LITE project demonstrated an important trade-off: stronger reliability behavior can coexist with reduced answerability, exact instruction following, response diversity, and useful directness when calibration becomes too narrow. The MERCURY report therefore tracks both what the model can answer and whether it answers within the correct behavioral and output contract.

This document is an internal development record, not a marketing brochure or an independent public benchmark claim.

3. MODEL AND DEVELOPMENT INVENTORY

Item	Value
Product name	SAYMTHEON MERCURY
Development line	6.5 / half-developing calibration phase
Base model	Mistral-7B-v0.3
Parameter count	7.248B
Weight precision	BF16
Maximum working context	3,500 tokens
Training examples	55,278 valid examples
Train / validation	54,173 / 1,105
Batch / gradient accumulation	1 / 16
Learning rate	1e-5
Epoch plan	2; early stopping
Selected checkpoint	checkpoint-3250
Best evaluation loss	0.5813245
Final model	SAYMTHEON-6.4-MERCURY-FINAL

4. TRAINING AND CHECKPOINT SELECTION

Validation loss improved consistently during training, reaching the best recorded value of 0.5813 at checkpoint-3250. Evaluation loss subsequently worsened twice, so training was stopped and checkpoint-3250 was selected as the final model state.

Stage	Eval loss	Status
250	0.7072	Initial evaluation
750	0.6880	Improving
1000	0.6778	Improving
1250	0.6650	Improving
2500	0.6078	Strong improvement
~2750	0.5984	Improving
~3000	0.5892	Improving
3250	0.5813	Best checkpoint
~3500	0.6039	First degradation
3750	0.6053	Second degradation / stop

The checkpoint decision follows the same principle established in the LITE development work: low training loss alone is not sufficient evidence for release quality. Behavioral evaluation remains part of checkpoint selection.

5. BENCHMARK METHODOLOGY

The main internal evaluation was originally generated as 1,500 benchmark records. After removing duplicate prompts, the scored set contains 1,286 unique prompts. All reported percentages in this document refer to that unique prompt snapshot.

Category	Unique	Direction
Core capability	50	Higher is better
Reliability	5	Higher is better
Security	100	Failure lower is better
Instruction following	100	Higher is better
Tool authority	100	Higher is better
Agent boundaries	100	Higher is better
RAG injection	100	Failure lower is better
False premise	100	Failure lower is better
Mathematics	100	Higher is better
Coding	100	Strict format
Systems / architecture	100	Higher is better
Reasoning	100	Higher is better
Long context	100	Requires separate interpretation
Response length	31	Failure lower is better
Repetition	100	Failure lower is better

The 0% long-context result must be interpreted together with the 3,500-token inference limit and benchmark design. It should not by itself be treated as definitive proof that long-context capability is absent.

6. CURRENT PERFORMANCE SNAPSHOT

Category	MERCURY	Interpretation
Core capability	98%	Very strong
Reliability	80%	Below target
Security failure	0%	Strong on failure metric
Instruction following	61%	Clear calibration gap
Tool authority	75%	Partial generalization
Agent boundaries	25%	Significant gap
RAG injection failure	0%	Strong on failure metric
False-premise failure	75%	Significant gap
Mathematics	0%	Calibration required
Coding strict	0%	Output-contract gap
Coding functional	100%	Functional generation strong
Systems / architecture	100%	Strong
Reasoning	0%	Calibration required
Long context	0%	Separate validation required
Response-length failure	93.5%	Major control issue
Repetition failure	0%	Strong on failure metric

Effective pass: $641 / 1,286 = 49.8\%$. The base model measured $171 / 1,286 = 13.3\%$ under the same internal scoring framework, a difference of +36.5 percentage points. This comparison is specific to the internal benchmark.

7. CURRENT STRENGTHS

- Core capability — 98%: strong performance across the general capability subset.
- Systems / architecture — 100%: strong performance on systems and architecture questions.
- Functional coding — 100%: functional code generation is strong even though strict-format compliance remains weak.
- Security failure — 0%: no failures were measured in the evaluated security-failure subset.
- RAG injection failure — 0%: strong separation between retrieved content and instruction authority.
- Repetition failure — 0%: no failures were measured in the repetition metric.

These strengths align with the reliability principles established during the LITE work: retrieved text should be treated as data rather than system authority, tool output should not automatically constitute authorization, and execution success should not be claimed without evidence.

8. KNOWN GAPS AND CALIBRATION AREAS

1 — Exact instruction following

The model can solve a task while still violating the requested output contract. Examples include code-only output, integer-only output, exact line limits, and strict word limits.

2 — Mathematics

The strict benchmark measured 0%. This does not by itself establish that mathematical capability is absent. Observed behavior indicates that the model can enter a meta-response mode instead of directly solving the problem.

3 — Reasoning

A similar response-mode issue is visible: the model can describe what an instruction is asking instead of performing the requested reasoning.

4 — Agent boundaries

The 25% score indicates that distinctions between planning, suggesting, authorizing, executing, and proving completion do not yet generalize reliably across all tested scenarios.

5 — False premises

The 75% failure rate indicates that unsupported premises are still accepted or processed instead of being consistently corrected.

6 — Response length

The 93.5% failure rate shows that output-length control is one of the most significant remaining behavioral problems.

9. LESSONS CARRIED FORWARD FROM LITE 4B

The LITE 4B project showed that reliability-oriented training can narrow the response space and make safe response patterns overly dominant. Its development record documented a trade-off between boundary reliability and useful answerability, instruction following, diversity, and directness.

MERCURY shows a different form of calibration problem: several safety and capability dimensions are already strong, but the model does not consistently select the correct response mode for exact-format tasks, mathematics, reasoning, false premises, agent boundaries, and response length.

The next phase should therefore be a small, controlled behavioral calibration rather than a large reliability-heavy retraining cycle.

Behaviors to preserve

- RAG injection resistance
- Security boundary behavior
- Systems / architecture capability
- Functional coding
- Repetition control

Behaviors to correct

- Exact instruction following
- Math/reasoning response mode
- False-premise correction
- Agent/tool authority generalization
- Response-length control

10. 6.5 CALIBRATION PLAN

The next phase is intended to correct behavioral selection rather than rebuild the model's capabilities. For mathematics and reasoning in particular, the first objective is to recover the correct solving mode before adding new knowledge.

Phase	Goal	Approach
A — Diagnose	Separate failure clusters	Math, reasoning, strict format, false premise, agent/tool, length
B — Corrective SFT	Behavioral calibration	Small high-quality corrective set; low learning rate; short run
C — Preservation	Protect existing strengths	Mix capability and boundary-preservation examples
D — Locked Eval	Regression control	Same 1,286 unique prompts; A/B comparison
E — BF16 Validation	Final verification	Validate the winning behavior on the original BF16 model

The central engineering rule is simple: the calibration pass should teach the model to select the correct behavioral mode, not to memorize the benchmark.

11. CURRENT MODEL STATUS

Dimension	Status
Model weights	Complete
Inference readiness	Complete
General capability	Strong
Systems / architecture	Strong
Functional coding	Strong
Security / RAG boundary	Strong
Instruction following	Calibration required
Math / reasoning mode	Calibration required
Agent boundaries	Further development required
False-premise handling	Further development required
Response length	Further development required
Long-context measurement	Separate validation required
Production readiness	Not complete

Half-developing definition: MERCURY is no longer only an experimental training artifact. It can be loaded, run, and benchmarked. However, behavioral calibration is not complete, so this checkpoint should not yet be labeled as the final production release.

12. CONCLUSION

SAYMTHEON MERCURY has reached a meaningful development checkpoint: the model can be loaded and executed, and it shows substantial movement over the base model on the internal 1,286-unique-prompt benchmark. Effective pass is 49.8%, compared with 13.3% for the base model under the same scoring framework.

The central remaining engineering problem is not simply adding knowledge. It is selecting the correct operating mode: solve the problem directly, follow the requested output contract, correct unsupported premises, distinguish tool or agent state from actual execution, and control the requested response length.

The LITE 4B work established the importance of balancing reliability with useful answerability. MERCURY's next phase carries that lesson forward into a larger model while preserving the reliability behaviors already demonstrated.

Current status: SAYMTHEON MERCURY — HALF-DEVELOPING / CALIBRATION PENDING.

Developed by SAYMTHEON.

APPENDIX A — TECHNICAL SCORE RECORD

Measurement	Result
Unique prompts	1,286
Effective pass	49.8%
Base effective pass	13.3%
Delta	+36.5 points
Core capability	98%
Systems / architecture	100%
Functional coding	100%
Instruction following	61%
Agent boundaries	25%
Mathematics	0%
Reasoning	0%
Response-length failure	93.5%
Repetition failure	0%

Failure metrics are better when lower. The 0% strict-coding score must be read together with the 100% functional coding result: the current evidence points more strongly to output-contract compliance than to an inability to produce code.

Developed by SAYMTHEON.